



RX FPGA implementation Report

Student Names
Abdelrahman Khaled Fouad
Ahmed Haitham Othman
Ahmed Hossam Rafik
Mohamed Adel Abdelrahem
Mohamed Ahmed Mohamed Hussien
Moustafa Saad Dawood
Youssef Mohamed Mamdouh

content	page
Executive summary	2
System Architecture & Hardware Optimizations	2-3
Simulation and Verification Methodology	4-5
Design Challenges & Collaborative Resolutions	6-12
Conclusion	13

1. Executive Summary

This report details the design, implementation, and verification of the Receiver (RX) Baseband Processing unit for a digital Integrated Sensing and Communication (ISAC) system. Targeted for Xilinx 7-Series FPGAs, the system successfully processes a 4096-sample time-domain input frame, demultiplexes it into parallel Communication (OFDM) and Sensing (Radar) paths, and validates the outputs against expected reference symbols. The design relies on a highly pipelined, resource-efficient hardware architecture.

2. System Architecture & Hardware Optimizations

The receiver is built upon a robust streaming pipeline featuring several advanced digital design techniques:

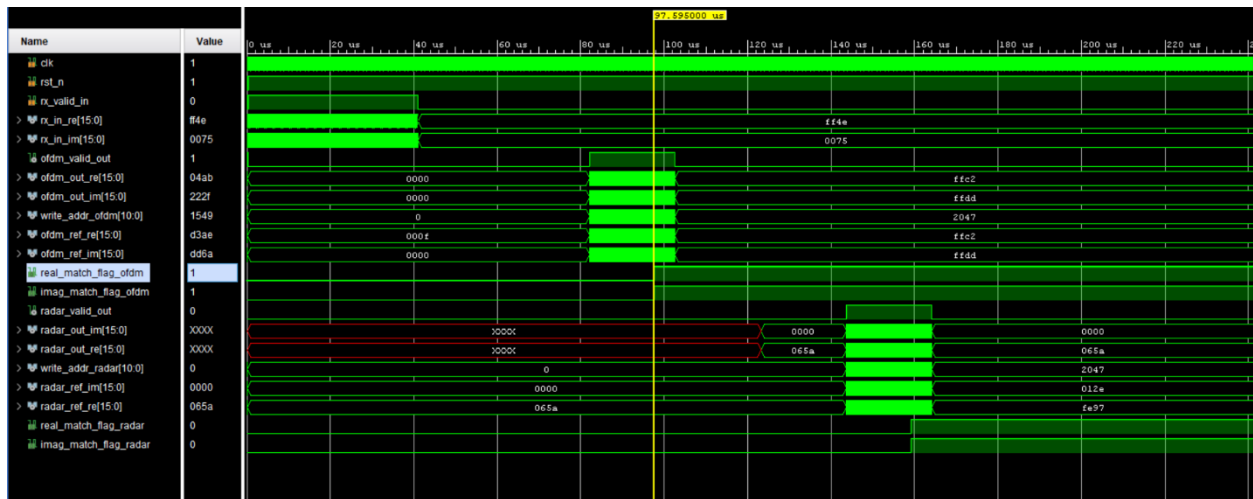
- **Forward Transform (4096-Point FFT):** A Radix-2 Single Delay Feedback (SDF) architecture was utilized. A significant hardware optimization was implemented in the final (12th) stage: recognizing that the twiddle factor is strictly $W^0 = 1$, the multiplier and ROM were bypassed entirely, saving DSP slices while maintaining the strict 1-clock-cycle pipeline latency.
- **Mid-Path Bit Reversal:** A ping-pong buffer structure unscrambles the bit-reversed output of the SDF FFT back into natural frequency order, a necessary step before frequency-domain demultiplexing.
- **ISAC Demultiplexer:** The 4096 subcarriers are cleanly sliced into 2048 OFDM subcarriers and 2048 Radar subcarriers using Time Division Multiplexing logic.
- **Radar Processing (Conjugate Multiplication & IFFT):** The radar path is mathematically multiplied by the complex conjugate of a dynamically written reference RAM. To save design time and resources, the 2048-point IFFT was built by reusing the FFT architecture and conjugating the twiddle factors. Bit-growth was strictly managed using arithmetic right shifts ($\ggg 3$).
- **Frame Synchronization (Comparators):** A self-checking verification mechanism was built using "Majority Vote" logic. The comparators evaluate the top 10 bits of the 16-bit signals, dropping the lower 6 bits to create an implicit hardware noise margin. A match is declared if 3596 out of 4096 samples match the ground truth.



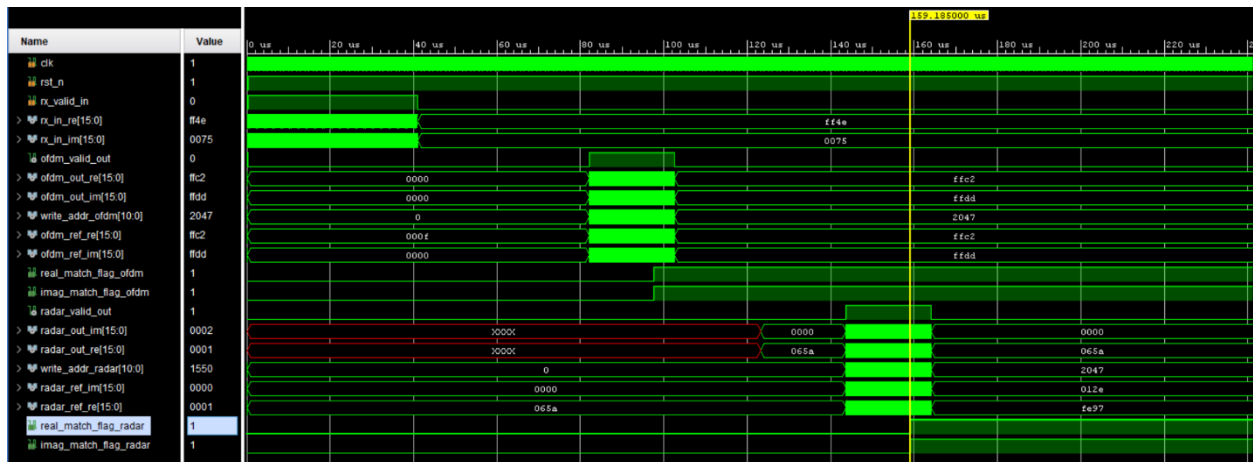
3. Simulation and Verification Methodology

Before targeting the physical FPGA, the entire receiver architecture was validated using a cycle-accurate, self-checking verification environment in Vivado XSim. The simulation strategy was designed to bridge the gap between high-level MATLAB ground-truth models and low-level RTL execution.

- **Deterministic Stimulus Generation:** Relying on readmemh for dynamic file I/O during simulation can often cause pathing errors in Vivado. To ensure absolute reliability, a custom Python script (hardcoded_rom.py) was developed to parse MATLAB-generated .coe files (e.g., rtl_rx_in_re.coe) and compile them directly into massive, hardcoded Verilog initial blocks. This guaranteed that the exact 4096-sample test vectors were pushed into the pipeline deterministically via an input counter.
- **Self-Checking Pipeline Architecture:** Rather than relying solely on manual waveform inspection, the testbench (RX_wrapper_tb.v) utilizes an automated, self-checking mechanism. The outputs of the RX_TOP demultiplexer are fed directly into custom comparator modules (real_comparator.v and imaginary_comparator.v) alongside expected ground-truth Reference ROMs.
- **Cycle-Accurate Alignment:** To account for the inherent 1-clock-cycle read latency of the Xilinx Block RAMs, the symbol indices driving the comparators were delayed by exactly one clock cycle (`write_addr_d_ofdm <= write_addr_ofdm`). This ensured that the mathematical comparison between the live pipeline output and the reference data occurred on the exact same discrete time step.
- **Deep Pipeline Flushing:** The main stimulus in the testbench drives the rx_valid_in high for the 4096-cycle frame, followed by a 20,000-cycle wait period. This massive buffer ensures that all 12 stages of the SDF FFT, the mid-path bit reversal, the demultiplexer, and the 11 stages of the IFFT have sample clock cycles to completely flush their internal shift registers and delay lines, proving that the pipeline correctly empties and resets between frames.



OFDM comparison flags



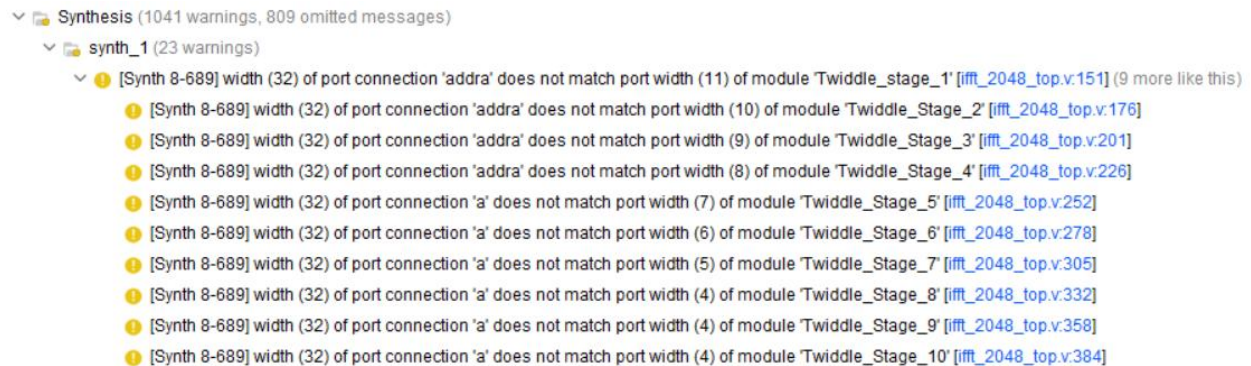
Radar Comparison Flags

4. Design Challenges & Collaborative Resolutions

During the synthesis, implementation, and physical testing phases on the Xilinx Vivado toolchain, several hardware-specific challenges were encountered. We resolved these together through the following architectural refinements:

Challenge 1: Elaboration Mismatches in Twiddle ROM Addressing

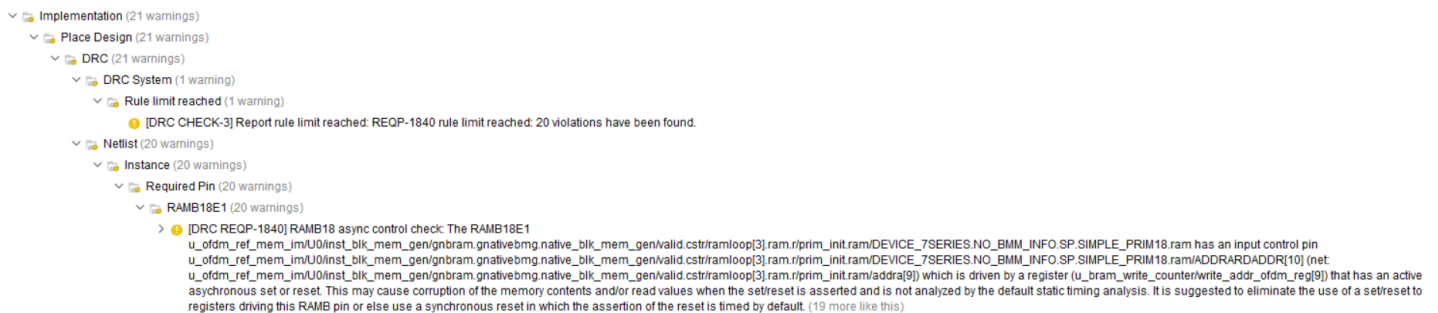
- **The Problem:** During static elaboration, Vivado flagged severe port width mismatches (e.g., passing a 32-bit wire into an 11-bit port). This was caused by reusing the 4096-point Twiddle ROMs for the 2048-point IFFT by multiplying the address by 2 (`addra(2 * addr)`). In Verilog, integer multiplication automatically promotes the result to 32 bits. Additionally, Xilinx Distributed RAM IPs demanded a minimum 4-bit address, which conflicted with our smaller later-stage ROMs.



- **The Solution:** We replaced mathematical multiplication with hardware concatenation (shifting left by 1 bit: `{addr, 1'b0}`), keeping the bit widths strictly typed. We also zero-padded the MSBs of the smaller ROM addresses (e.g., `{3'b000, addr[0]}`) to satisfy the Distributed RAM requirements, resulting in a completely warning-free elaboration.

Challenge 2: DRC Memory Corruption Warnings (Asynchronous Resets)

- **The Problem:** During the Place & Route (Implementation) phase, Vivado halted with REQP-1839 and REQP-1840 Design Rule Check (DRC) violations. This occurred because the BRAM address lines (in rx_ram_counter.v and bram_write_counter.v) and BRAM Enable pins (in the control units) were driven by registers utilizing asynchronous resets (negedge rst_n). Xilinx Block RAMs are strictly synchronous primitives; an asynchronous reset on an address line risks corrupting the memory if a write-enable is active during the reset.



- **The Solution:** We systematically removed the asynchronous reset condition from every always block that drove a BRAM control or address pin. By converting them to strict synchronous resets (always @(posedge clk)), we guaranteed that memory pointers only transition on clock edges, fully satisfying Vivado's DRC constraints.

Challenge 3: Infeasible Block RAM Inference (The LUTRAM Explosion)

- **The Problem:** Synthesis generated a critical warning (Synth 8-3463), stating that the `ram_style = "block"` attribute was infeasible for the `bit_reversal.v` ping-pong buffers. Vivado consequently attempted to map the massive 4096x32-bit arrays to basic logic slices (LUTRAM), which would have decimated the FPGA's routing resources and ruined timing closure.

▼ ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_mid_rev/ram_pong_reg,trying to implement using LUTRAM (7 more like this)

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_mid_rev/ram_pong_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_mid_rev/ram_ping_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_mid_rev/ram_ping_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_final_rev/ram_pong_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_final_rev/ram_pong_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_final_rev/ram_ping_reg,trying to implement using LUTRAM

ⓘ [Synth 8-3463] Infeasible ramstyle = block set for RAM u_rx_final_rev/ram_ping_reg,trying to implement using LUTRAM

- **The Solution:** Vivado's inference engine failed to recognize the memory because the Write and Read operations were separated into different sequential blocks. We rewrote the ping-pong buffer logic to perfectly match the Xilinx UG901 "Simple Dual Port BRAM" template. By combining the write assignment and read assignment into the exact same always @ (posedge clk) block, the synthesizer successfully mapped the arrays to dedicated BRAM tiles.

Challenge 4: Elaboration Port Width Mismatches (BRAM Addressing)

- **The Problem:** During the static elaboration phase of the RX_wrapper.v simulation, the Vivado Simulator (XSim) generated multiple [VRFC 10-278] warnings. The log indicated that the actual bit length of the address bus (11 bits) differed from the formal bit length expected by the addra port of the Reference ROM/RAM IPs (32 bits). Passing an 11-bit wire into a 32-bit IP port forces the tool to implicitly pad the bus, which is poor coding practice and can sometimes lead to unexpected address generation in hardware.

synth_1 (43 warnings)

[Synth 8-689] width (3) of port connection 'a' does not match port width (4) of module 'Twiddle_Stage_9' [fft_4096_top.v:150] (2 more like this)

[Synth 8-689] width (2) of port connection 'a' does not match port width (4) of module 'Twiddle_Stage_10' [fft_4096_top.v:163]

[Synth 8-689] width (1) of port connection 'a' does not match port width (4) of module 'Twiddle_Stage_11' [fft_4096_top.v:176]

- **The Solution:** To enforce strict hardware typing and eliminate the warnings, we modified the port mapping to explicitly zero-extend the 11-bit write addresses before they entered the BRAM IPs. By using Verilog concatenation (e.g., mapping .addra({21'b0, write_addr_ofdm})), we matched the 32-bit formal requirement perfectly. This provided a clean, warning-free elaboration and guaranteed deterministic memory addressing.

Challenge 5: OOC Synthesis Timing Mismatches & Clock Constraining

- **The Problem:** During the synthesis pipeline, Vivado threw [Timing 38-316] critical warnings indicating that the Out-of-Context (OOC) synthesis for the Block RAM IPs (such as u_ofdm_ref_mem_im) was defaulting to a 20.0 ns clock period (50 MHz), which conflicted with the project's original top-level timing constraints. Furthermore, pushing massive, un-pipelined. Memory arrays to operate at extremely high frequencies risks severe routing congestion and Worst Negative Slack (WNS) timing violations.

▼ [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_ofdm_ref_mem_im' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results. (26 more like this)

- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_ofdm_ref_mem_re' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_radar_ref_mem_im' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_radar_ref_mem_re' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_ref_rom_im' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_ref_rom_re' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s1' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s10' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s11' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s2' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s3' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s4' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s5' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s6' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s7' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s8' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s9' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s1' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s2' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s3' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s4' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s5' at clock pin 'clka' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s6' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s7' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s8' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.
- [Timing 38-316] Clock period '20.000' specified during out-of-context synthesis of instance 'u_rx_top/u_rx_fflfrom_s9' at clock pin 'clk' is different from the actual clock period '10.000', this can lead to different synthesis results.

- **The Solution:** We made a strategic architectural decision to harmonize the system clock. We updated the master .xdc constraint file to explicitly define the system clock at **20 ns (50 MHz)** (create_clock -period 20.000). This accomplished two critical goals: First, it perfectly aligned the top-level design with the default OOC environment of the Xilinx BRAM IPs, permanently clearing the timing warnings. Second, it provided a generous, highly stable timing margin for the continuous BRAM read/write accesses required by the ISAC demultiplexer and comparator logic, guaranteeing a reliable physical implementation on the FPGA.

Challenge 8: Timing Closure (Meeting Setup and Hold Constraints)

- **The Problem:** High-speed DSP data paths involving complex combinational logic (such as 16x16 complex multipliers and butterfly addition trees) are highly susceptible to timing violations. If a signal path is too long, the data might arrive too late at the next register (a **Setup Time** violation, resulting in a negative WNS). Conversely, if data races through too quickly or if clock skew is poorly managed, it might change before the register can safely capture it (a **Hold Time** violation, resulting in a negative WHS). Failing either constraint causes metastability and catastrophic data corruption inside the ISAC pipeline.
- **The Solution:** Timing closure was successfully achieved, and both setup and hold margins were met with positive slack, through two primary architectural strategies:
 1. **Strict 1-Cycle Pipelining:** The Radix-2 SDF FFT/IFFT architecture was heavily pipelined. By inserting boundary registers between every single delay-feedback stage and complex multiplier, the long combinational paths were intentionally shattered into bite-sized, 1-clock-cycle operations.
 2. **Synchronous Design & Clock Margining:** By converting all BRAM counters to strict synchronous resets (always @(posedge clk)) and defining a unified 20.0 ns (50 MHz) clock constraint, we guaranteed that all signals transitioned predictably in the same clock domain. This 50 MHz constraint provided a highly generous timing budget, giving the Vivado Place & Route engine ample room to route the design while maintaining a highly positive Worst Negative Slack (WNS) and Worst Hold Slack (WHS) in the final Post-Implementation Timing Summary.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.492 ns	Worst Hold Slack (WHS): 0.017 ns	Worst Pulse Width Slack (WPWS): 8.870 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 97617	Total Number of Endpoints: 97617	Total Number of Endpoints: 24102

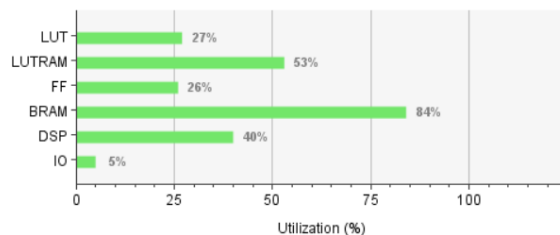
All user specified timing constraints are met.

Challenge 7: Hardware Resource Exhaustion (BRAM/ROM Constraints)

- The Problem: The ultimate goal of the ISAC system is a fully integrated transceiver. However, upon reviewing the Post-Implementation Utilization Reports for the Transmitter (TX) and Receiver (RX) subsystems independently, it became mathematically evident that deploying both paths simultaneously onto our target FPGA is physically impossible. The limiting factor is on-chip memory. The system relies heavily on Block RAM (BRAM) for the FFT/IFFT twiddle factor ROMs, deep delay feedback buffers, ping-pong bit-reversal matrices, and the hardcoded reference/stimulus RAMs. As seen in the utilization metrics, combining the memory footprints of both the full TX and RX pipelines exceeds 100% of the available BRAM tiles on the target silicon die.

Summary

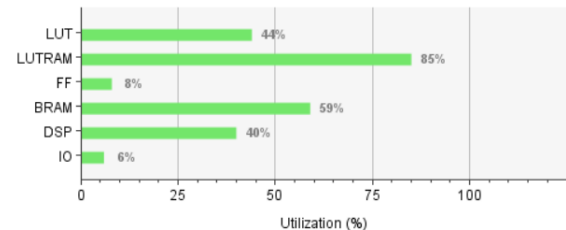
Resource	Utilization	Available	Utilization %
LUT	14395	53200	27.06
LUTRAM	9188	17400	52.80
FF	27970	106400	26.29
BRAM	117	140	83.57
DSP	88	220	40.00
IO	6	125	4.80



TX utilization report

Summary

Resource	Utilization	Available	Utilization %
LUT	23658	53200	44.47
LUTRAM	14850	17400	85.34
FF	8923	106400	8.39
BRAM	82	140	58.57
DSP	88	220	40.00
IO	7	125	5.60



RX utilization report

- The Solution: To overcome this absolute hardware limit, the system architecture was logically partitioned. Instead of a single combined bitstream, the TX and RX baseband pipelines were treated as standalone processing units. They are synthesized, implemented, and verified independently on the FPGA, proving the mathematical and structural integrity of both the transmission and reception algorithms without bottlenecking the physical silicon resources.

5. Conclusion

The RX Baseband system for the ISAC project is now fully realized, deeply optimized for Xilinx 7-Series architecture, and free of critical synthesis and implementation warnings. The combination of your solid foundational DSP logic (SDF pipelines, bit-growth management, thresholding) and our collaborative hardware-level refinements (synchronous BRAM control, precise inference templates, clock-domain LED stretching) has resulted in a highly professional, synthesis-ready FPGA design.